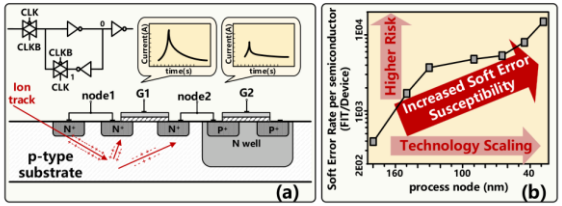
面向数字电路可靠性提升的 SEU故障分析与加固优化方法研究

相关工作调研: DMBF (代理模型 + 贝叶斯优化)
与 ZACHO (零样本深度强化学习加固)

汇报材料

1 DMBF: Design Metrics Balancing Framework for Soft-Error-Tolerant Digital Circuits Through Bayesian Optimization, IEEE TCASI, 2023

2 ZACHO: Zero-Shot Agile Circuit Hardening Optimizer via Deep Reinforcement Learning, IEEE TVLSI, 2026



软错误机理与工艺缩放下的 SER 变化趋势

ZACHO, IEEE TVLSI 2026 / Fig. 1

研究对象

寄存器位翻转 (SEU) 的故障溯源与设计阶段选择性加固

CONTENTS

目录

01 **研究背景与问题界定**

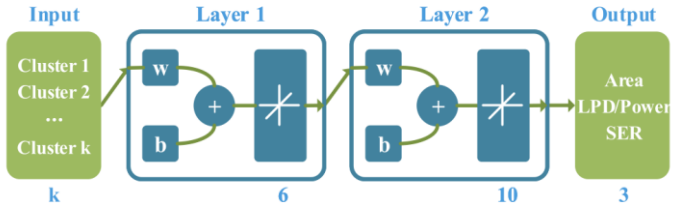
SEU 机理与屏蔽效应 / 加固对象选择 / 2^N 配置空间与配置相关收益

02 **DMBF: 代理模型与贝叶斯优化**

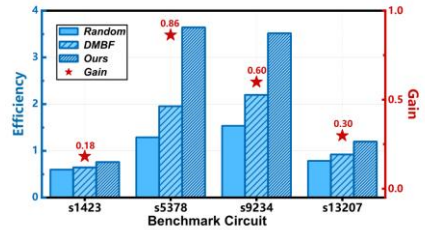
IEEE TCASI 2023 / 数据预处理降维 + 贝叶斯神经网络 + 多目标贝叶斯优化

03 **ZACHO: 零样本深度强化学习加固**

IEEE TVLSI 2026 / 代理模型辅助的 A2C 强化学习, 支持未见电路零样本泛化



DMBF Fig.6 贝叶斯神经网络代理模型结构



ZACHO Fig.9 各基准电路的加固效率与 SER 增益

研究背景: SEU 机理与加固对象的选择

软错误的三条产生路径

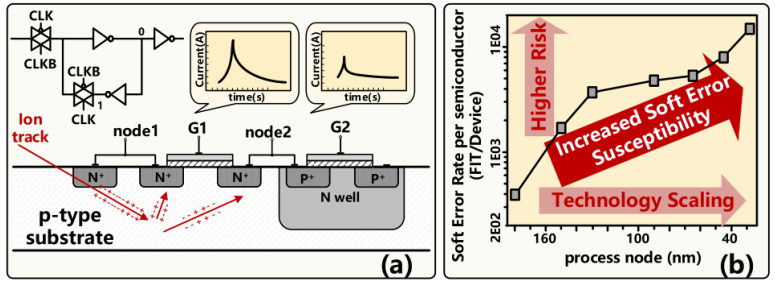
高能粒子击中敏感节点产生瞬态电压: 时序单元被击中形成 SEU, 组合逻辑被击中形成 SET

三种固有屏蔽机制

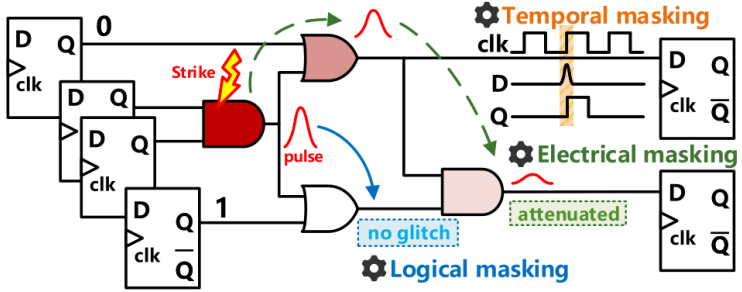
电气屏蔽 / 逻辑屏蔽 / 时序屏蔽只能间歇性生效, 因此仍需设计级加固手段

寄存器是首要加固目标

触发器与 SRAM 具有状态保持特性, 是电路级选择性加固的首选加固对象



ZACHO Fig.1 (a) 软错误产生机理 (b) 工艺缩放下的 SER 变化



ZACHO Fig.2 组合路径上的瞬态电压演化与三种屏蔽效应

问题: 全部加固带来面积 / 功耗代价, 选择性加固的难点在于加固收益与面积开销随当前配置变化

01

DMBF: 多目标贝叶斯优化的软错误容错电路设计

数据预处理降维 + 贝叶斯神经网络代理模型 + 多目标贝叶斯优化 (MOBO)

IEEE Transactions on Circuits and Systems-I, Vol.70, No.10, 2023 | Yan Li, Chao Chen, Xu Cheng, Jun Han, Xiaoyang Zeng (Fudan University)

GLOBAL MAP

两个阶段

阶段一　数据预处理

聚类 -> 排序 -> 实数编码

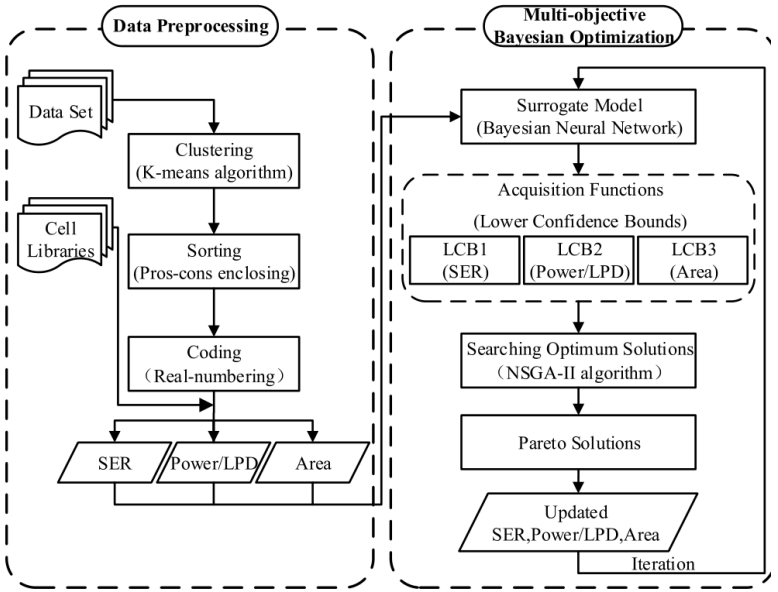
把 n 维离散配置压成 k 维连续变量, 让大规模电路变得可优化

阶段二　多目标贝叶斯优化

BNN 代理 + LCB 采集 + NSGA-II 选点

每一轮只花一次真实仿真预算, 换来一点信息, 更新模型, 再评估

以加固单元 (如 TMR) 替换原有单元, 必然引入面积 / 时延 / 功耗开销, 因此形成多目标优化 (MOO) 问题: 在替换方案空间中寻找 SER 下降与实现代价的平衡点. 设计指标与电路单元之间呈黑盒关系, 直接求解代价高昂.
关键: 昂贵的是"真跑一次仿真", 不是"算一次代理模型". 全套设计的唯一目的就是少花那笔预算



DMBF Fig.2 框架流程图: 右侧竖列为阶段二的闭环, 底部为数据回灌

THE PROBLEM

起点: 为什么这个问题不能硬算

2^{5315}

c5315 含 5315 个逻辑门, 每个门独立决定加不加固, 就是这么多种方案

病一 高维

工业电路含百万至十亿个单元, 解空间 2^n, 计算复杂度与存储开销 "unacceptable", 直接求解 "may be impossible"

病二 离散

x_i 只能取 0 或 1, 而贝叶斯优化这类方法主要面向连续变量, 离散优化问题至今仍缺乏成熟算法

原始优化问题 (论文式 8)

$$\min \quad f(x) = [\, f_1(x), f_2(x), f_3(x) \,]^T$$

$$\text{s.t.} \quad x_i \in \{0, 1\}, \quad i = 1 \dots n$$

$x = [x_1, x_2, \dots, x_n]^T$

$x_i = 1$ 第 i 个单元换成加固版本 (TMR / BISER 等)

$x_i = 0$ 该单元保持不变

论文原话: *the MOO problem in (8) is not trivial to address for two reasons: large dimensionality and discrete form.*

所以阶段一必须同时做两件事: 降维 (n -> k) 与离散转连续

STAGE 1

阶段一: 三步操作, 解决两件事

1 K-means 聚类

按"加固代价/收益"把单元分组, 同簇内单元几乎等价

得到: k 个簇

2 Pros-Cons Enclosing 排序

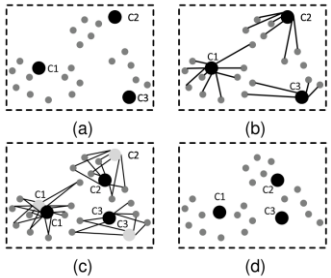
每簇内部排成一条队: 先加固谁最划算, 排在前面

得到: 有序数组

3 实数编码

每簇的 0/1 位串换成"加固了几个"这一个实数

得到: k 维连续输入



DMBF Fig.3 k-means 的四步实现

阶段一解决的两件事

降维 5315 个单元 -> k 个簇

连续化 0/1 位串 -> 连续实数

论文原话: "dramatically reduces the data dimensionality (i.e., the number of replacement schemes) and transforms the discrete optimization problem into a continuous one"

STAGE 1 · STEP 1

① K-means: 聚的不是"功能相似", 是"加固性价比相似"

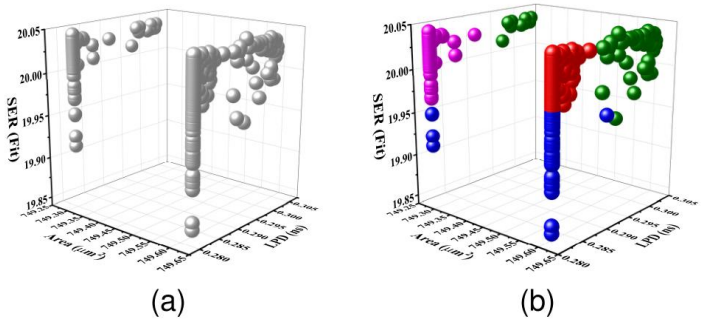
每个单元的 3 维向量是什么

= 只加固这一个单元时, 整个电路的三个指标

- Area 越小 = 加固的面积代价越小
- LPD / 功耗 越小 = 加固的时延代价越小
- SER 越小 = 加固它降掉的软错误越多 (收益越大)

为什么全都可以"越小越好"

三个维度的方向一致, 所以才能加权合成一个"临界因子" F 去排序. 也正因为同簇单元的代价与收益都差不多, 它们才可以互相替代 -- 这正是后面"一个实数代表一簇"能成立的前提.



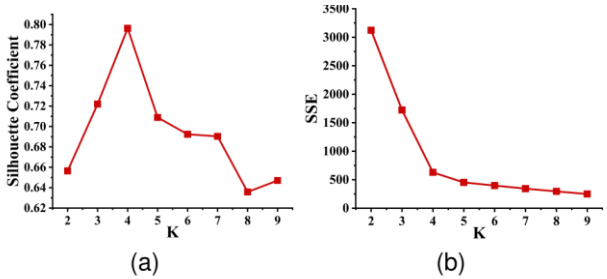
DMBF Fig.8 c5315 聚类结果: (a) 原始点云 (b) 聚类后着色

看图的两个细节

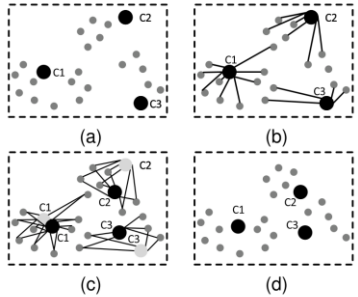
- (a) 原始点云不是均匀的云, 而是几根竖直的"柱子" -- 同类型同尺寸的门坐标几乎重合
- (b) 4 种颜色各自抱团, 即 k = 4, 与 SC/SSE 定出的结果一致

STAGE 1 · STEP 1

① k 怎么定, 以及聚类具体怎么跑



DMBF Fig.7 c5315: (a) 轮廓系数 SC (b) SSE 随 k 的变化



DMBF Fig.3 k-means 实现: (a) 随机质心 (b) 分配 (c) 重算 (d) 收敛

两个指标一起看

SC 在 k = 4 取极大值, 同时 SSE 恰好出现明显肘点 -> 定 k = 4

s9234 (时序) 同样是 k = 4

优化目标: 最小化簇内平方和 WCSS (式 14)

$$\min \sum_j \sum_{x \in C_i} \| x - \mu_i \|^2$$

一个前提不能忽略: 面积约 749, SER 约 20, LPD 约 0.3, 量级差三个数量级, 论文在聚类前先做 z-score 归一化, 否则欧氏距离完全被面积主导

STAGE 1 · STEP 2

② Pros-Cons Enclosing: 把"先加固谁"算清楚

临界因子 (式 15)

组合电路 $F_cc = w_SER*SER + w_area*area + w_LPD*LPD$

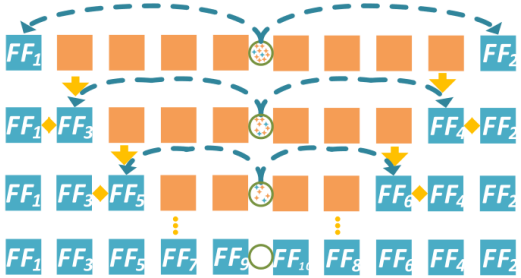
时序电路 $F_sc = w_SER*SER + w_area*area + w_power*power$

F 小 => 面积小, 时延/功耗小, SER 小 => 该单元最该加固

Algorithm 2 的四步

- F 最小的单元当队首, F 最大的当队尾
- 每轮从剩余元素里, 取离当前队首最近的一个填头部, 取离当前队尾最近的一个填尾部
- 递归重复, 直到整条数组填满
- "Enclosing" 指的就是这个过程: 两端向中间生长, 把中间围起来

最终是关键单元在前段, 次要单元在后段



DMBF Fig.4 (上半) Pros-Cons Enclosing 排序过程

排序解决的是"优先级", 不是"选谁"

排序只保证: 当你决定加固这一簇时, 先加固哪几个最划算. 至于"这一簇到底加不加固, 加几个", 留给下一阶段的实数编码去决定.

STAGE 1 · STEP 1

Table I

Inv	Random			Sorted		
	Area (μm^2)	LPD (ns)	SER (Fit)	Area (μm^2)	LPD (ns)	SER (Fit)
0	749.616	0.3017	20.0054	749.616	0.2944	19.9529
1	749.616	0.3005	20.0306	749.616	0.2985	19.9749
2	749.616	0.3015	20.040	749.616	0.2975	19.9935
3	749.616	0.2924	20.033	749.616	0.2964	20.0128
4	749.616	0.3014	20.0318	749.616	0.3007	20.0069

DMBF Table I c5315 某一个簇在排序前后的原始指标值

SER 列

Sorted 里 SER 随位置单调上升: 19.9529 -> 20.0069. 越靠前, 加固它之后电路剩下的 SER 越小 = 降掉的软错误越多 = 收益越大, 所以排最前.

LPD 列

越靠前 LPD 越小 (0.2944 vs 0.3007), 说明加固它对时序的拖累最小. 加固代价与收益同时最小, 才会排在最前.

Area 列

簇内几乎恒定 (749.616), 说明同簇单元的加固代价相同, 可以互相替代 -- 这就是实数编码成立的原因.

STAGE 1 · STEP 2

② 排序的真正价值: 让数据变平滑

平滑度指标 ASD (式 19)

$$ASD = sum | K(i+1) - K(i) | / (N - 1)$$

K_i 是第 i 个点的斜率. 直线的 ASD = 0, 所以 ASD 越小, 数据变化越平滑连续.

为什么平滑这么重要

- 阶段二要在连续空间上用代理模型拟合指标曲面
- 数据跳变剧烈, 代理模型就难拟合准, 采集函数也容易挑到坏点
- 排序把"按敏感度渐进加固"的序列铺平, 让代理模型好学, 让 BO 的选点更准

论文原话: "the sorting operation makes the data more suitable for the subsequent multi-objective Bayesian optimization and promotes the accuracy of the model"

TABLE II

ASD RESULTS OF ARRAY BEFORE AND AFTER SORTING

Category	Random	Sorted
SER&Area	0.01739	0.01184 (**31.91%** ↓)
SER&LPD	1.83802	0.98060 (**46.65%** ↓)

DMBF Table II c5315 排序前后的 ASD 对比

排序前后 ASD 的变化

SER & Area 0.01739 -> 0.01184 (-31.91%)

SER & LPD 1.83802 -> 0.98060 (-46.65%)

时序电路 s9234 同理: SER&Area -44.08%, SER&Power -40.00% (Table V)

STAGE 1 · STEP 3

③ 实数编码

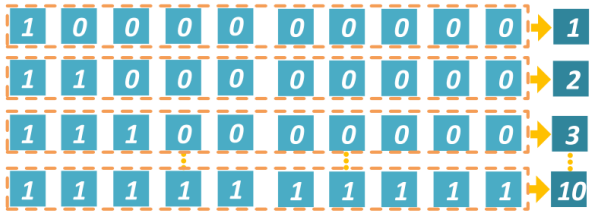
编码规则 (论文原话)

把 "1" 连续排在前段 (类比温度计编码), 然后用数组里 "1" 的个数作为这一簇的编码值, 把离散二进制变成连续十进制数.

编码的语义

1 1 0 0 0	→ 2.0	加固最划算的那 2 个
1 1 1 1 0	→ 4.0	加固前 4 个
1 1 1 1 1	→ 5.0	全加固

语义: 从 0 到 m, 就是"按敏感度从高到低渐进加固". 排序已经把顺序算好了, 编码只是把它变成一个可以连续调的数.



DMBF Fig.4 (下半) 温度计式位串与实数编码: 1, 2, 3, 10

阶段一的最终成果

输入: 5315 个 0/1 位 (2^5315 种)

变成: k 个连续实数 (c5315 上 k = 4)

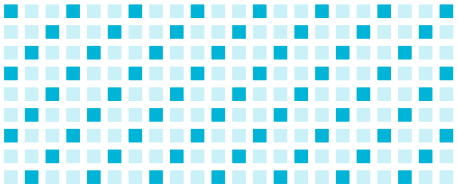
输出: 三个设计指标 (面积 / 时延或功耗 / SER). 于是式 8 的离散问题变成式 16 的连续问题.

STAGE 1

阶段一: 拿到了什么, 付出了什么

降维的直观画面

5315 个单元, 各自独立决定加不加固



2^5315 种组合, 每一组都要跑一次综合 + 故障仿真

聚成 4 簇



4 个连续值, 贝叶斯优化终于可以在低维连续空间里工作

拿到了什么

- 维度 n -> k, 离散 -> 连续, 大规模电路变得可优化
- 同一簇内单元可互相替代, 决策空间被真正压小
- 排序让"先加固谁"变成可计算量, 而不是凭经验

付出了什么

- 簇内个体差异被抹平: 同一簇里"具体加固哪一个"的决策消失了
- 权重 w_SER / w_area / w_LPD 要靠人定, 改一下权重, 排序就变
- 问题退化为"每簇加固几个", 而非"加固哪些"

STAGE 2

阶段二: 贝叶斯优化的"评估-更新"闭环

1 随机初始化

随机撒一批点 D0, 真评估, 训出第一个 BNN

2 产生候选

NSGA-II 在 k 维连续空间演化种群, 给出候选点 x

3 打分

BNN 输出折成三个 LCB

4 非支配排序

NSGA-II 按三个 LCB 求 POS

5 真跑一次

从 POS 里挑一个点, 跑综合 + 故障仿真

唯一昂贵的一步

6 回灌重训

真实结果加入数据集, 重训 BNN, 回到第 2 步

回灌重训: 每评估一个点, 模型就更新一次, 下一个点是在"最新认知"上挑的. 算法 1 的注释写得很直白 -- Augment data, Update probabilistic surrogate model. 没有这个环, BO 就只是"一次拟合加一次猜测".

BO 到底在优化什么

不在"求一个最优解", 而在用最少的真实仿真次数把 Pareto 前沿探出来. 最后交付给设计师的是一族可选方案.

为什么必须多轮

第一轮的 LCB 几乎不可信 (数据太少, 方差巨大), 只能当"瞎猜 + 探索"用; 数据累积后 LCB 才逐渐贴近真实边界.

每轮只花一次仿真

如果每轮把整个 POS 都评估一遍, 开销 = 种群规模 x 轮数, BO 的样本效率优势直接归零, 那还不如直接跑 NSGA-II.

STAGE 2

职责分离: NSGA-II 生成候选, BNN 只负责打分

生成器

NSGA-II

在 k 维连续空间里演化种群, 产生候选点 x.

候选点不是 BNN 给的, 是 NSGA-II 自己的种群给的.

打分器

BNN

对任意候选 x, 输出后验分布 {u_i, sigma_i^2}, i = 1,2,3.

选择器

LCB + NSGA-II

把每个目标的 (mu_i, sigma_i^2) 压成 1 个 LCB, 于是 6 个数变 3 个目标.

再由 NSGA-II 做非支配排序, 输出 POS.

为什么中间必须有 LCB 这一步? NSGA-II 需要 3 个目标才能跑三目标 MOO, 而 BNN 给的是 6 个数 (三个目标的 mu 和 sigma^2 各一个). LCB 的作用就是**把每个目标的 (mu_i, sigma_i^2) 这一对压成一个数**, 压完正好 3 个.

论文原话: "the optimization has turned into an MOO problem for the LCB functions" / "NSGA-II is applied to solve the multi-objective problem for the three LCB functions"

STAGE 2

LCB: 三个目标各一个, 兼顾"猜得好"与"我没把握"

$$LCB_i(x) = beta_i * sigma_i^2(f_i(x)) - mu_i(f_i(x))$$

i = 1 (SER), 2 (功耗/LPD), 3 (面积); 三个目标各自独立构造, 目标是最小化

mu_i(x) — **我猜这方案有多好**

BNN 预测的指标均值. 越小说明预测这方案越划算.

sigma_i^2(x) — **我有多没把握**

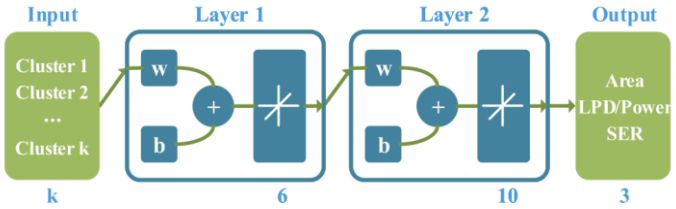
BNN 预测的方差. 越大说明这块地方模型还没学好, 值得去看一眼.

beta_i — **好奇心的刻度**

在"利用"与"探索"之间权衡. 论文里是预设常数, 不随迭代自动调整.

一句话直觉

最小化 LCB = 同时偏好"预测值好"和"我完全没底".
所以 LCB 天生是探索器, 不是收敛器 -- 它挑出来的不是最优解, 而是"最值得去试"的方向.



DMBF Fig.6 BNN 结构: 权重服从后验分布, 不确定度就是这么来的

为什么选 LCB 而不是 UCB: LCB 形式简单, 且不需要给定参考点 -- 而 GP 的 UCB 需要参考点, 这对 MOO 很麻烦

SUMMARY

总结

1. 阶段一: 聚类把单元按"加固性价比"分组 (等价类), 排序把"先加固谁"算清楚, 实数编码把每簇压成一个连续旋钮
2. 阶段二: NSGA-II 生成候选, BNN 打分, LCB 把 6 个数压成 3 个目标, 再由 NSGA-II 求 POS, 每轮只挑一个点做真实验证
3. 全程只有一件事是昂贵的: 真跑一次综合 + 故障仿真. 降维与平滑排序, 都是为了让这笔预算花得更省

02

ZACHO: 零样本深度强化学习的敏捷电路加固

代理模型辅助的 A2C 强化学习, 在合成电路上训练并零样本泛化到未见电路

IEEE Transactions on Very Large Scale Integration (VLSI) Systems, Vol.34, No.7, 2026 | Junfeng Zhu, Yueming Hu, Liang Wang, Zhe Li, Yan Li, Xiaoyang Zeng

GLOBAL MAP

全局地图: 离线训练一次, 在线零样本部署

训练期 · 离线 **数据生成**

造合成电路实例 + 资源预算, 归一化成 100 万条训练集

训练期 · 离线 **代理模型**

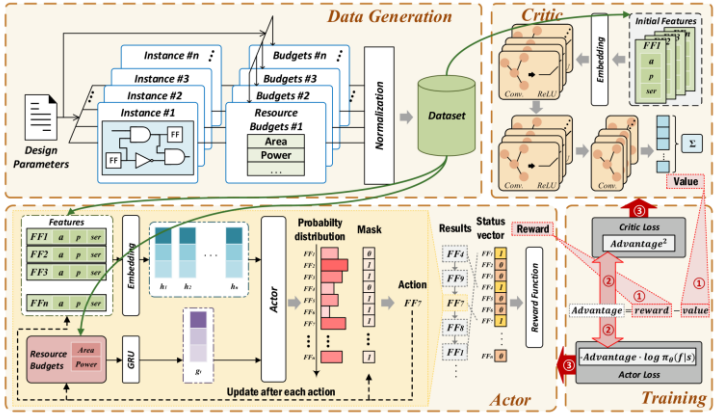
PCA + 全连接网络, 学会代替 DC/BFIT 快速预测三个指标

训练期 · 离线 **A2C 训练**

Actor 学"该选哪个 FF", Critic 估值做基线, 联合更新

推理期 · 在线 **零样本部署**

真实电路上不重训, 逐个选 FF, 代理模型在线重算特征漂移



ZACHO Fig.5 框架总览: 数据生成 / Actor / Critic / Training 四个模块

THE CORE IDEA

ZACHO：从组合优化到序贯决策

DMBF: 一次性交答卷

min f(x) = [f1(x), f2(x), f3(x)]^T x in {0,1}^N

- 给一整条加固清单, 评价它好不好
- 评价与顺序无关 -- 清单里 FF 的排列不影响结果

ZACHO: 每一步挑一个

max SUM(t=0..T-1) Dser(f_t, t)

- 式 (9) 的求和下标是 t -- 收益是逐步累加的
- 每一步的选择依赖前面已经做的决定
- 于是问题变成: 在资源约束下的序贯决策

为什么这个改写很关键

加固 f3 之后网表变了, 综合重新优化, **f5 的性价比也跟着变了**. 论文原话: "the modifications in FF feature representations induced by logic synthesis optimization exhibit inherent complexity and unpredictability. Consequently, decisions that appear locally optimal at individual steps may fail to converge toward a globally optimal solution." -- 每一步看着最优的选择, 走到底可能不是全局最优.

特征漂移

收益随当前配置变化, 必须每步重算

硬约束掩码

可行性由机制保证, 不靠策略学得好

可迁移的策略

学的是"怎么选", 而非"这个电路多好"

TRAINING · STEP 1

训练期: 数据生成 -- 为什么用合成电路

合成规则 (论文原文)

每个 FF 的特征从**预定义均匀分布独立采样**, 保证统计多样性

初始资源预算 = 所有 FF 特征总和的**某个比例**, 而这个比例本身也是采样出来的

全部特征按采样出的预算做归一化, 保证不同实例的特征尺度一致

为什么必须合成

- ZACHO 要的是零样本泛化 -- 换任何电路都不用重训
- 要让策略见过各种规模 / 各种预算松紧 / 各种特征分布的组合
- 真电路只有 4 个 benchmark, 远远不够
- 代价: 策略是在人造分布上学的, 真电路上还要靠代理模型兜底

TABLE I

BASELINE METRICS OF THE EVALUATED ISCAS'89 BENCHMARK CIRCUITS

Circuit	FF Number	Power (mW)	Area (μm^2)	SER (FIT)
s1423	74	0.3583	464.02	18404
s5378	179	0.9245	1091.16	33187
s9234	211	0.9365	1351.73	103888
s13207	638	2.8065	2804.59	171856

ZACHO Table I 四个 ISCAS89 基准电路的基线指标

真实评估设置 (Table II)

训练集 **1,000,000** 验证集 **1,000** 训练轮数 **4**

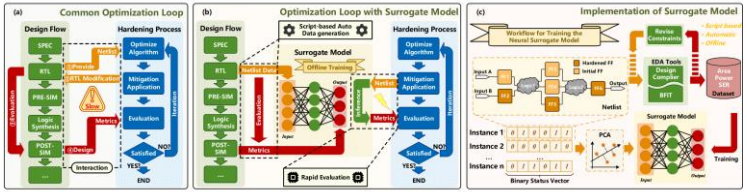
隐藏维 **128** batch **256** dropout **0.1** GRU 层数 **1**

Actor / Critic 学习率均为 5e-4, 最大梯度范数 2.0

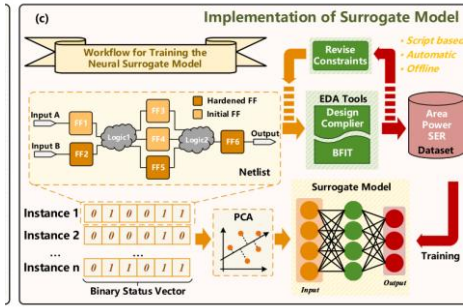
这一步决定的是"策略能不能泛化": 数据越多样, 零样本迁移越稳, 但也越依赖代理模型在真实电路上的能力

TRAINING · STEP 2

训练期: 代理模型 -- 把 EDA 变成一次 MLP 推理



ZACHO Fig.3 (a) 传统优化回路: 每步都跟 Design Flow 交互 (b) 引入代理模型后 EDA 评估被替代 (c) 代理模型自身的实现流程



ZACHO Fig.3(c) 代理模型三阶段: 数据生成 -> PCA -> 训练 -> 部署

数据生成	随机 0/1 向量 -> 改网表 -> DC 算面积/功耗, BFIT 算 SER
模型训练	PCA 保留 70% 方差 -> z-score -> 全连接单隐层, 训到 train/val 精度 > 95%
部署	优化期间完全替代 EDA, 毫秒级推理

精度: 面积 > 99.9%　功耗 > 99.2%　SER 95.0% - 98.3%

TRAINING · STEP 3

训练期: A2C 训练, 以及 Critic 的特殊设计

Algorithm 1 Advantage Actor–Critic Batch Training

Input: Actor network π_θ, Critic network V_ϕ, dataset $\mathcal{D}$, reward function $R(\cdot)$, batch size B, learning rates α_θ, α_ϕ

Output: Optimized parameters θ, ϕ

Initialize parameters θ, ϕ ;

for *epoch* $= 1$ **to** E **do**

  foreach *batch* $\{s_i\}_{i=1}^{B}$ *sampled from* $\mathcal{D}$ **do**

    Sample action sequence $a_i \sim \pi_\theta(s_i)$;

    Evaluate rewards: $r_i \leftarrow R(s_i, a_i)$;

    Estimate values: $\hat{v}_i \leftarrow V_\phi(s_i)$;

    Compute advantages: $A_i \leftarrow r_i - \hat{v}_i$;

    Compute actor loss:

$$\mathcal{L}_{\text{actor}} = -\frac{1}{B}\sum_{i=1}^{B} A_i \cdot \log \pi_\theta(f_i|s_i) \ ;$$

    Compute critic loss: $\mathcal{L}_{\text{critic}} = \frac{1}{B}\sum_{i-1}^{B}(A_i)^2$;

ZACHO Algorithm 1 Advantage Actor-Critic 批量训练

每一步在算什么

4: 采样动作序列 a_i ~ pi_theta(s_i)

5: 执行得到奖励 r_i = R(s_i, a_i)

6: Critic 估值 v_i = V_phi(s_i)

7: 优势 A_i = r_i - v_i

8: L_actor = -(1/B) SUM A_i * log pi_theta(f_i | s_i)

9: L_critic = (1/B) SUM (A_i)^2

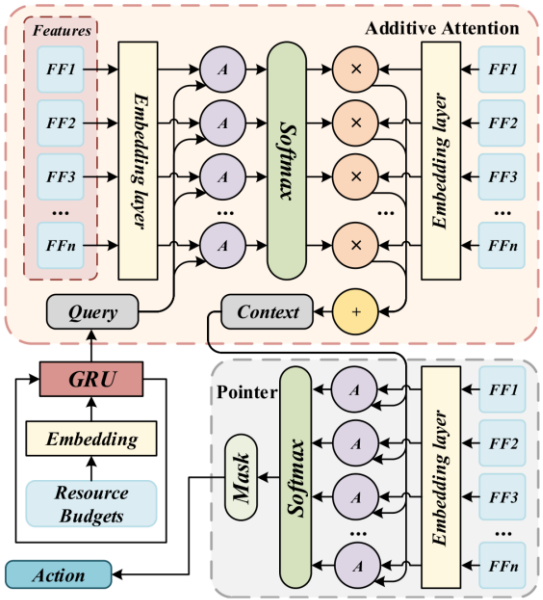
Critic 为什么只看初始状态

V_phi(X_0) = SUM(i=1..N) f_phi(x_i,0)

- 只吃初始特征 X_0, 一次前向就算完
- 对 FF 顺序置换不变 (求和), 所以 N 变化也能泛化
- 不需要建模长时序依赖, 计算开销极小
- 它只是个打分员, 用来给 Actor 的每一步提供基线, 降低策略梯度方差

INFERENCE · ONE STEP

推理期: 一步决策内部到底发生了什么



ZACHO Fig.6 单步数据流

1 编码

每个 FF 的 [area, power, ser] 过共享 1-D 卷积 -> h_i (式 15); 预算向量 L_t 走另一条编码器 -> b_t (式 16)

2 上下文

GRU 用 (b_t, g_t-1) 累积"还剩多少预算 + 之前怎么选的"; Query 与每个 h_i 拼接后打分 -> 注意力权重 alpha_i -> 加权求和得 Context c_t (式 18-21)

3 指针选择

每个 h_i 与 c_t 拼接, 用另一组参数算选择能量 e_i (式 22); softmax 后叠加掩码 m_i -> 概率分布 (式 23)

4 掩码与执行

已选过的 / 会超预算的 FF 掩码置 -inf, softmax 后概率精确为 0; 训练时采样, 推理时取 argmax; 选完更新状态向量 / 预算 / 特征 / 掩码

关键: 注意力是每一步重新分配的 -- 上一步选了什么, 预算剩多少, 直接改变这一步"该看谁"

INFERENCE · WORKED EXAMPLE

推理期: 特征漂移 -- 加固一个 FF 会改变其他 FF 的性价比

真实电路 s9234: 211 个 FF, SER 103888 FIT, 面积 1351.73 um2, 功耗 0.9365 mW (Table I)

t = 0 初始状态

x(i,t) = [Darea, Dpower, Dser] （式 5）

FF	Darea	Dpower	Dser
f1	12	8	0.30
f2	20	15	0.22
f3	**9**	**5**	**0.38**
f4	25	20	0.18
f5	14	9	0.26
f6	30	25	0.12

t = 1 加固 f3 之后 (代理模型重算)

同一批 FF, 三个量全都变了

FF	Darea	Dpower	Dser
f1	**17**	**11**	**0.26**
f2	20	15	0.22
f4	25	20	0.18
f5	**10**	**7**	**0.30**
f6	30	25	0.12

这一步发生了什么

- 策略选中 f3 (收益 0.38 只花 9 面积, 效率最高)
- 预算更新 (式 7-8): L_1 = { 100-9, 60-5 } = { 91, 55 }
- 代理模型按新状态重算所有剩余 FF 的增量 -> 上面右表

为什么这很要命

- f1: 面积代价涨了, 边际收益却降了 -- f3 抢掉了一部分软错误路径
- f5: 反而变便宜了, 收益还变大了 -- 综合重优化改写了它的逻辑锥
- 方向完全相反. 按初始敏感度排好的序, 到这一步已经失效了

INFERENCE · WORKED EXAMPLE

推理期: 掩码怎么把约束变成硬保证

t = 0
初始
L0 = { 100, 60 }
选 f3
L1 = { 91, 55 }

t = 1
漂移后
L1 = { 91, 55 }
选 f5
L2 = { 81, 48 }

t = 2
漂移后
L2 = { 81, 48 }
选 f1
L3 = { 64, 37 }

t = 3
漂移后
L3 = { 64, 37 }
选 f2
L4 = { 42, 21 }

t = 4
终止
L4 = { 42, 21 }
可行集为空
T = 5

t = 4 掩码生效, 可行集为空

剩余预算 L4 = { 42, 21 }

FF	Darea	Dpower	Dser	判定
f4	28	22	0.15	超预算
f6	34	40	0.10	超预算

f4 的 Dpower 22 > 21, f6 的 Dpower 40 > 21 -- 两者都被掩码, 概率精确为 0, 终止.

掩码: 约束是硬保证

m(i,t) 属于 { -inf, 0 }

p(i,t) = exp(e(i,t) + m(i,t)) / SUM(j) exp(e(j,t) + m(j,t))

- 掩码加在 softmax 之前, 所以被屏蔽的 FF 概率精确为 0
- 约束不是"希望策略学得好", 而是机制上不可能违反
- 对比 DMBF: 靠优化器在 Pareto 前沿里自己挑满足约束的点, 论文也承认约束可能被违反

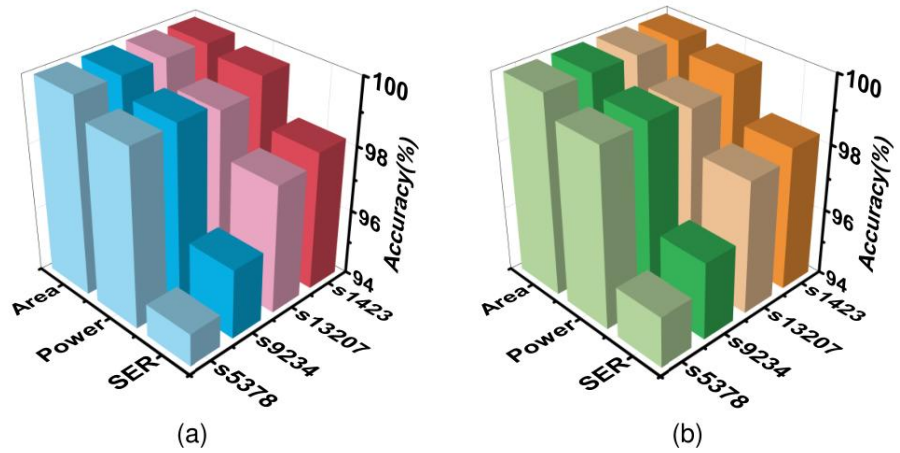
最终序列 f3 -> f5 -> f1 -> f2, 累计 Dser = 0.38 + 0.30 + 0.24 + 0.20 = 1.12

COMPARISON

DMBF 与 ZACHO: 到底换了什么

维度	DMBF（IEEE TCASI 2023)	ZACHO（IEEE TVLSI 2026)
数学形式	组合优化: min f(x), x 属于 {0,1}N	**序贯决策: max SUM(t) Dser(f_t, t)**
谁选下一个	NSGA-II 在 4 维旋钮空间里搜	**Actor 网络逐步从 N 个 FF 里指针式选**
收益随配置变化	不处理: 排序 + 人工权重把先验固化	**每步重算特征 + 注意力动态重分配**
约束满足	靠优化器在 Pareto 前沿里自己挑, 可能违反	**掩码, 机制上不可能违反**
不确定性利用	方差进 LCB, 但 beta 是预设常数	**注意力每步动态分配**
换电路	数据集作废, 模型作废, 重新跑	**零样本, 不重训（代理模型还是得重新训练）**
并行性	BO 串行, 浪费并行集群	序贯决策, 同样串行
产出	一族 Pareto 解	一族 Pareto 解 + 给定预算下的直接最优

实验一: 代理模型的预测精度



ZACHO Fig.7 代理模型精度: (a) 训练集 (b) 测试集

论文报告的精度

Area > 99.9%

Power > 99.2%

SER 95.0% - 98.3%

SER 最难: FIT 由 FF 脆弱性和电路级相互依赖决定, 强非线性

论文自己的提醒

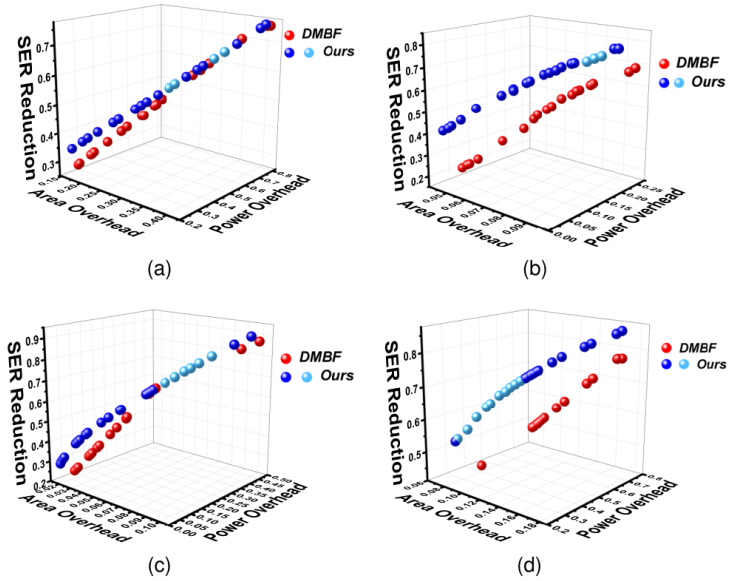
"for scenarios requiring strict resource constraints, we recommend introducing an appropriate safety margin based on the surrogate model's prediction accuracy and the target resource limits."

翻译: 严格约束场景要按代理模型的精度加安全裕量, 否则预测误差可能让最终方案违反约束

这一点直接对应你方案里要报告的"约束违例率"与"真实满足面积约束时的 SER": 代理模型准不准, 最终方案就靠不靠谱

论文还做了 10 次不同随机种子的重复训练: s5378 上 SER 精度训练集均值 95.25%, 测试集 95.18%, 最大差距 < 0.4%

实验二: Pareto 前沿 -- 为什么逐个决策能铺出更细的折中



ZACHO Fig.8 四个基准电路的三维 Pareto 前沿: (a) s1423 (b) s5378 (c) s9234 (d) s13207

怎么看这张图

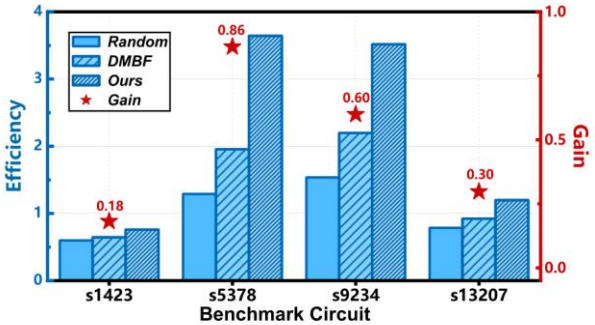
- 红 = DMBF, 蓝 = ZACHO (Ours)
- ZACHO 的点明显更密, 沿着前沿连续排开
- DMBF 的点稀疏, 且在目标空间有明显断裂

为什么会这样

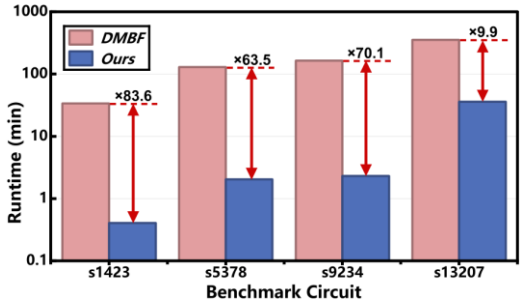
- DMBF 是"一次定一整簇的旋钮", 方案数受旋钮粒度限制
- ZACHO 逐个选 FF, 每加一个就产生一个新的中间状态
- 细密的决策路径天然铺出细密的折中曲线
- *论文原话: "the Pareto front produced by DMBF is sparse and exhibits noticeable discontinuities in the objective space"*

量化指标: ZACHO 的平均超体积 (HV) 比 DMBF 高 19.58%; HV 越大, 说明覆盖的目标空间越广, 收敛性与多样性越好

实验三: 加固效率与运行时间加速



ZACHO Fig.9 各基准电路的加固效率与相对随机基线的增益



ZACHO Fig.10 生成整条 Pareto 前沿所需的总运行时间 (分钟, 对数轴)

s1423	s5378	s9234	s13207
x83.6	**x63.5**	**x70.1**	**x9.9**
加速比	加速比	加速比	加速比

加速来自哪里

- DMBF 每轮都要重训 BNN, 还要反复调 LCB 采集函数
- ZACHO 只做一次策略前向 + 一次代理模型推理, 换电路直接用
- 论文强调加速的是推理时间, 不是训练时间

全部结果由单一模型生成, 无需重训或针对电路做调整; 论文中的加速比区间为 9.9x 至 83.6x